1 数组
1.1 理论基础
1.1.1 存储方式
1.1.2 二维数组
1.2 经典题目
1.2.1 二分查找
例题：leetcode 704 - 二分查找
例题：leetcode 35 - 二分查找
例题：leetcode 34 - 在排序数组中查找元素的第一个和最后一个位置
1.2.2 快慢指针
例题：leetcode 977 - 有序数组的平方
例题：leetcode 209 - 长度最小的子数组
例题：leetcode 904 - 水果成篮
例题：leetcode 76 - 最小覆盖子串
1.2.3 模拟生成
例题：leetcode 59 - 螺旋矩阵II
例题：leetcode 54 - 螺旋矩阵
1.2.4 前缀和
例题：kamacoder 58 - 区间和
例题：kamacoder 44 - 开发商购买土地
1.3 常用函数总结

1 数组

1.1 理论基础

- 数组是非常基础的数据结构，在面试中，考察数组的题目一般在思维上都不难，主要是考察对代码的掌控能力

1.1.1 存储方式

- 首先要知道数组在内存中的存储方式，这样才能真正理解数组相关的面试题
    - **数组是存放在连续内存空间上的相同类型数据的集合。**
- 需要两点注意的是
    - **数组下标都是从0开始的。**
    - **数组内存空间的地址是连续的**
- 正是**因为数组在内存空间的地址是连续的，所以我们在删除或者增添元素的时候，就难免要移动其他元素的地址。**
- 而且大家如果使用C++的话，要注意vector 和 array的区别，vector的底层实现是array，严格来讲vector是容器，不是数组。
- **数组的元素是不能删的，只能覆盖。**

1.1.2 二维数组

- **二维数组在内存的空间地址是连续的么？**

- 不同编程语言的内存管理是不一样的，以C++为例，在C++中二维数组是连续分布的。

1.2 经典题目

1.2.1 二分查找

例题：leetcode 704 - 二分查找

- 给定一个 `n` 个元素有序的（升序）整型数组 `nums` 和一个目标值 `target` ，写一个函数搜索 `nums` 中的 `target`，如果 `target` 存在返回下标，否则返回 `-1`。你必须编写一个具有 `O(log n)` 时间复杂度的算法。

[704. 二分查找 - 力扣（LeetCode）](https://leetcode.cn/problems/binary-search/)

```cpp
class Solution_704{
public:
    int search(vector<int>& nums, int target) {

        int n = nums.size();
        int left = 0, right = n - 1;

        while(left <= right){

           int arrow = (left + right) / 2;

           if(nums[arrow] < target){
                left = arrow + 1;
                arrow = (left + right) / 2;
           }

           else if(nums[arrow] > target){
                right = arrow - 1;
                arrow = (left + right) / 2;
           }

           else return arrow;
        }

        return -1;
    }
};
```

- 二分法最有可能出错的地方一般发生在循环判断停止的时候。我们一般采取一个左闭右闭的区间对其进行二分查找，利用双指针分别记录最左侧和最右侧的查找指针范围，每次取半与目标值进行大小比对，注意到当我们缩小二分区间的时候，需要将区间收缩的比之前更紧（这意味着：左侧收缩时，需要对边界值加一，而右侧收缩时，需要对边界值减一）
    - 如果不这样做会发生什么？这会导致出现中间指针总是会等于left或者right中的某一个，既不好判断目标值，也无法停止循环。

例题：leetcode 35 - 二分查找

- 给定一个排序数组和一个目标值，在数组中找到目标值，并返回其索引。如果目标值不存在于数组中，返回它将会被按顺序插入的位置.请必须使用时间复杂度为 `O(log n)` 的算法。

[35. 搜索插入位置 - 力扣（LeetCode）](#)

```
class Solution_35 {
public:
    int searchInsert(vector<int>& nums, int target) {

        int n = nums.size();
        int left = 0, right = n - 1;
        int arrow = (left + right) / 2;

        while(left <= right){

            if(nums[arrow] < target){
                left = arrow + 1;
                arrow = (left + right) / 2;
            }

            else if(nums[arrow] > target){
                right = arrow - 1;
                arrow = (left + right) / 2;
            }

            else return arrow;
        }

        if(right < 0) return 0;
        if(nums[arrow] < target) return left;
        else return right;
    }
};
```

- 和上道题目略有不同的是，需要输出未包含在原数组中的数字的可能存在的编号位置，这就带来了一个挑战：当我们循环停止时得到了一个left>right的两个数，那么应该将哪一个作为新的数的下标呢？其实需要分类讨论，如果target大则插在left上，否则插在right上，注意到还有一个坑点：
    - **left和right可能越界，left可能上溢到n，这个不用管，right可能下溢到-1，这个需要直接调整为0**（不用判断是因为right下溢到-1时，nums[0]一定大于target了）

例题：leetcode 34 - 在排序数组中查找元素的第一个和最后一个位置

- 给定一个按照升序排列的整数数组 nums，和一个目标值 target。找出给定目标值在数组中的开始位置和结束位置。
    - 如果数组中不存在目标值 target，返回 [-1, -1]。
    - 进阶：你可以设计并实现时间复杂度为 $O(\log n)$ 的算法解决此问题吗？

[34. 在排序数组中查找元素的第一个和最后一个位置 - 力扣（LeetCode）](#)

```cpp
class Solution_34 {
public:
    int fl(vector<int>& nums, int target){
        int n = nums.size();
        int left = 0,right = n-1;
        while(left<right){
            int mid=(left+right)/2;
            if(nums[mid] >= target) right = mid;
            else if(nums[mid] < target) left = mid+1;
        }
        return left;
    }
    int fr(vector<int>& nums, int target){
        int n = nums.size();
        int left = 0,right = n-1;
        while(left<=right){
            int mid=(left+right)/2;
            if(nums[mid] <= target) left = mid+1;
            else if(nums[mid] > target) right = mid-1;
        }
        return right;
    }
    bool f(vector<int>& nums, int target){
        int n = nums.size();
        int left = 0,right = n-1;
        while(left<=right){
            int mid=(left+right)/2;
            if(nums[mid] < target) left = mid+1;
            else if(nums[mid] > target) right = mid-1;
            else return true;
        }
        return false;
    }

    vector<int> searchRange(vector<int>& nums, int target) {
        vector<int> res(2,-1);
        if(nums.size() == 1 && nums[0] == target) return {0,0};
        if(f(nums,target)){
            res[0] = fl(nums,target);
            res[1] = fr(nums,target);
        }
        return res;
    }
};
```

- 注意到题目所叙述的是首先是一个可以重复出现多次的序列去查找首尾，所有需要考虑到三种情况：
    - 是否存在
    - 找到左侧边界
    - 找到右侧边界

- 我们针对以上的三种情况要分别进行讨论，第一种情况就类似于上述题目。我们直接写一个简单的函数就可以验证，注意到可以写成验证完是否存在之后再进行左右边界的计算，这样计算量会小很多。
- 左侧和右侧边界的计算需要分开讨论，注意一个**极容易犯错的地方**：
    - 在寻找右侧边界的时候，mid最后会因为四舍五入的缘故和left重合，所以如果逻辑写的是验证此处小于等于就去修改右端点等于mid的话就会陷入死循环，应该修正为mid-1，保证不会卡住。
- 最后还有一个小点：**如果采用的是小于作为停止条件的话，要注意数组长度为1时需要单独讨论。**

1.2.2 快慢指针

例题：leetcode 977 - 有序数组的平方

- 给你一个按 **非递减顺序** 排序的整数数组 `nums`，返回 **每个数字的平方** 组成的新数组，要求也按 **非递减顺序** 排序。
    - **进阶：** 请你设计时间复杂度为 `O(n)` 的算法解决本问题

[977. 有序数组的平方]

```
class Solution {
public:
    vector<int> sortedSquares(vector<int>& nums) {
        int length = nums.size();
        int start = 0;
        while(nums[start] < 0 && start <length - 1) start ++;
        int right = start,left = start - 1;
        vector<int> p2(length),res;
        for(int i=0;i<length;i++){
            p2[i] = nums[i]*nums[i];
        }
        while(left>=0 && right <length){
            if(p2[left]<=p2[right]) {
                res.push_back(p2[left]);
                left--;
            }
            else {
                res.push_back(p2[right]);
                right++;
            }
        }
        if(left == -1){
            for(int i = right;i<length;i++) res.push_back(p2[i]);
        }
        if(right == length){
            for(int i = left;i>=0;i--) res.push_back(p2[i]);
        }
        return res;
    }
};
```

- 注意到可能出现大小交错是因为有正负性的区别，越靠近0的数其平方数越小：

- 故我们首先查找最小的非负数作为交界点，安排两个指针分别向右和向左扫描，每次比较一个负数和正数平方大小（因为相同正负性的数大小已经排好），最后将剩余的数按次序插入即可。

例题：leetcode 209 - 长度最小的子数组

- 给定一个含有 `n` 个正整数的数组和一个正整数 `target` 。
  - 找出该数组中满足其总和大于等于 `target` 的长度最小的 **子数组** `[numsl, numsl+1, ..., numsr-1, numsr]` ，并返回其长度。如果不存在符合条件的子数组，返回 `0` 。

[209. 长度最小的子数组]

```
class Solution {
public:

    int minSubArrayLen(int target, vector<int>& nums) {
       int start = 0,length = 0,sum = 0;
       int result = 100001;
       for(int j=0;j<nums.size();j++){
            sum += nums[j];
            while(sum >= target){
                result = result < j - start + 1 ? result : j - start + 1;
                sum -= nums[start];
                start ++ ;
            }
       }
        return result == 100001 ? 0:result;
    }
};
```

- 这道题如果暴力计算的话就是枚举所有的子序列，显然不合适：
  - 我们采用滑动窗口的方法，让快指针不停向前，直到子序列的和已经比目标值更大，此时再让慢指针走，直到子序列的和比目标值小为止，此时记录下长度后，再让快指针向前直到比目标值大，以此不断往复，最后得到最小的一个长度输出。

例题：leetcode 904 - 水果成篮

- 你正在探访一家农场，农场从左到右种植了一排果树。这些树用一个整数数组 `fruits` 表示，其中 `fruits[i]` 是第 `i` 棵树上的水果**种类** 。

  你想要尽可能多地收集水果。然而，农场的主人设定了一些严格的规矩，你必须按照要求采摘水果：
  - 你只有 **两个** 篮子，并且每个篮子只能装 **单一类型** 的水果。每个篮子能够装的水果总量没有限制。
  - 你可以选择任意一棵树开始采摘，你必须从 **每棵** 树（包括开始采摘的树）上 **恰好摘一个水果** 。采摘的水果应当符合篮子中的水果类型。每采摘一次，你将会向右移动到下一棵树，并继续采摘。
  - 一旦你走到某棵树前，但水果不符合篮子的水果类型，那么就必须停止采摘。

  给你一个整数数组 `fruits` ，返回你可以收集的水果的 **最大** 数目。

[904. 水果成篮]

```
class Solution {
```

```
public:
    int totalFruit(vector<int>& fruits) {
        int length = fruits.size();
        int slow = 0,res = 0;
        unordered_map<int,int> barket;
        for(int fast = 0;fast<length;fast++){
            barket[fruits[fast]]++;
            while(barket.size()>2){
                auto it = barket.find(fruits[slow]);
                --(it -> second);
                if(barket[fruits[slow]] == 0) barket.erase(it);
                slow++;
            }
            res = max(res,fast-slow+1);
        }
        return res;
    }
};
```

- 这是一道快慢指针的扩展问题，当我们需要同时观测两个数值是否出现，并且摒弃第三值出现的时候，一个常见的想法是用哈希表去存储数值类型，若是出现第三种数值，则从左向右移动慢指针直到完全排除一个值为止。
    - 这里运用到了哈希表的一些常用函数：
        - find()：通过查询键，返回一个键值对。
        - It -> second：指的是键值对中的第二个元素也就是值。
    - 本题目的难点也就是在哈希表的运用上。

例题：leetcode 76 - 最小覆盖子串

- 给定两个字符串 `s` 和 `t`，长度分别是 `m` 和 `n`，返回 s 中的 **最短窗口子串**，使得该子串包含 `t` 中的每一个字符（**包括重复字符**）。如果没有这样的子串，返回空字符串 `""`。

  测试用例保证答案唯一。

[76. 最小覆盖子串]

```
class Solution {
public:
    unordered_map<char,int> exact,test;
    bool check(){
        for(auto &it:exact){
            if(it.second > test[it.first]) return false;
        }
        return true;
    }

    string minWindow(string s, string t){
        for(char it : t) exact[it]++;
        int l = 0, r = -1;
        int len = INT_MAX, ansL = -1,ansR = -1;
```

```

        while(r<int(s.length())){
            if(exact.find(s[++r])!=exact.end()) test[s[r]]++;
            while(check() && l<=r){
                if(r-l+1<len){
                    len = r-l+1;
                    ansL=l;
                }
                if(exact.find(s[l])!=exact.end()) --test[s[l]];
                l++;
            }
        }
        return ansL == -1? "":s.substr(ansL,len);
    }
};
```

- 本题目依旧采用快慢指针的思路，但关键在于如何判断子串是否满足题目条件，解答运用了两个哈希表分别存储目标子串和当前子串，如果符合要求就缩短，不符合就增长直到符合条件再缩小。

1.2.3 模拟生成

例题：leetcode 59 - 螺旋矩阵II

- 给你一个正整数 `n` ，生成一个包含 `1` 到 `n2` 所有元素，且元素按顺时针顺序螺旋排列的 `n x n` 正方形矩阵 `matrix` 。

[59. 螺旋矩阵 II]

```
class Solution {
public:
    vector<vector<int>> generateMatrix(int n) {
        vector<vector<int>> matrix(n, vector<int>(n,0));
        int up = 0, down = n-1, left = 0, right = n-1;
        int num = 1;
        while(up<down && left<right){
            for(int i = left;i<right;i++){
                matrix[up][i] = num;
                num++;
            }
            for(int i = up;i<down;i++){
                matrix[i][right] = num;
                num++;
            }
            for(int i = right;i>left;i--){
                matrix[down][i] = num;
                num++;
            }
            for(int i = down;i>up;i--){
                matrix[i][left] = num;
                num++;
            }
            left++;right--;up++;down--;
```

```
        }
        if(n%2==1) matrix[(n-1)/2][(n-1)/2] = n*n;
        return matrix;
    }
};
```

- 首先思考如果是手写应该会怎么去实现整个过程呢？就是按照顺时针从头写到尾，代码完全按照这个思路首先就可以：先横再纵再横再纵，然后缩小一圈再写一轮...直到上下左右边界相交，然后再考虑最后的情况，发现需要把技术情况单独考虑最后的尾值比较方便。

例题：leetcode 54 - 螺旋矩阵

- 给你一个 `m` 行 `n` 列的矩阵 `matrix` ，请按照 **顺时针螺旋顺序** ，返回矩阵中的所有元素。

[54. 螺旋矩阵]

```
class Solution {
public:
    vector<int> spiralOrder(vector<vector<int>>& matrix) {
        int m = matrix.size();
        int n = matrix[0].size();
        vector<int> res;
        int count = 0;
        int up = 0, down = m-1, left = 0, right = n-1;
        while(true){
            for(int i=left;i<=right;i++){
                res.push_back(matrix[up][i]);
                count++;
            }
            up++;
            if(up>down || count == m*n) break;
            for(int i=up;i<=down;i++){
                res.push_back(matrix[i][right]);
                count++;
            }
            right--;
            if(right<left|| count == m*n) break;
            for(int i=right;i>=left;i--){
                res.push_back(matrix[down][i]);
                count++;
            }
            down--;
            if(up>down || count == m*n) break;
            for(int i=down;i>=up;i--){
                res.push_back(matrix[i][left]);
                count++;
            }
            left++;
            if(right<left|| count == m*n) break;
        }

        return res;
```

```
    }
};
```

- 类似于上一道题的思路，但这个矩阵可能不是方的，所以再讨论时我们需要判定停止条件，也就是当有一对边超纲的时候就可以停止了，而不是像方阵一样可以等到四条边算完再停止，注意停止条件取不取等可以有不同的算法，本题和上一道题目就是两种不同的算法。

1.2.4 前缀和

例题：kamacoder 58 - 区间和

- 给定一个整数数组 Array，请计算该数组在每个指定区间内元素的总和。

[卡码58.区间和]

```
#include<iostream>
#include<vector>
using namespace std;
int main(){
    int n;cin >> n;
    vector<int> num(n);
    for(int i=0;i<n;i++) cin >> num[i];
    vector<int> sum(n+1);
    sum[0] = 0;
    for(int i=1;i<=n;i++) sum[i] = sum[i-1]+num[i-1];
    int a,b;
    while(cin>>a>>b){
        cout << sum[b+1] - sum[a]<< endl;
    }
    return 0;
}
```

- 这里运用到了非常经典的前缀和思路：我们如果每次单独求区间的数值和都要求一次和的话，显然速度会很慢，那我们就提前算一个数组，利用迭代遍历的流程求出前i项和，这样如果要求第a到b项的区间和，那么我们就只需要用前b项和减去前a-1项的和就行了。

例题：kamacoder 44 - 开发商购买土地

- 在一个城市区域内，被划分成了n * m个连续的区块，每个区块都拥有不同的权值，代表着其土地价值。目前，有两家开发公司，A 公司和 B 公司，希望购买这个城市区域的土地。
    - 现在，需要将这个城市区域的所有区块分配给 A 公司和 B 公司。
    - 然而，由于城市规划的限制，只允许将区域按横向或纵向划分成两个子区域，而且每个子区域都必须包含一个或多个区块。 为了确保公平竞争，你需要找到一种分配方式，使得 A 公司和 B 公司各自的子区域内的土地总价值之差最小。
- 注意：区块不可再分。
    - 给定一个整数数组 Array，请计算该数组在每个指定区间内元素的总和。

[卡码44.开发商购买土地]

```cpp
#include<iostream>
#include<vector>
#include<climits>
using namespace std;
int main(){
    int n,m;
    cin >> n >> m;
    int d[n][m];
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            cin >> d[i][j];
        }
    }

    int r[n]={0},c[m]={0};
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            r[i]+=d[i][j];
            c[j]+=d[i][j];
        }
    }

   int row[n],col[m];
   row[0] = r[0]; col[0] = c[0];
   for(int i=1;i<n;i++){
    row[i] = row[i-1]+r[i];
   }
   for(int i=1;i<m;i++){
    col[i] = col[i-1]+c[i];
   }

    int min = INT_MAX;
    for(int i=0;i<n;i++){
        if(abs(2*row[i]-row[n-1])<min){
            min = abs(2*row[i]-row[n-1]);
        }
    }
    for(int i=0;i<m;i++){
        if(abs(2*col[i]-col[m-1])<min){
            min = abs(2*col[i]-col[m-1]);
        }
    }
    cout << min << endl;
    return 0;
}
```

- 这道题目的关键点在于只能和横向或者竖向切分，所以分别求行和列的前缀和并选择最小的即可。

1.3 常用函数总结

以下是 C++ 中一维 `vector` 常用的函数及其说明：

1. 初始化与构造

```
vector<int> v1;                // 空向量
vector<int> v2(5);             // 5个元素（默认值0）
vector<int> v3(5, 42);         // 5个元素，初始化为42
vector<int> v4{1, 2, 3};       // 初始化列表 {1,2,3}
vector<int> v5(v4);            // 复制构造
```

2. 元素访问

函数	说明	示例
`operator[]`	下标访问（不检查边界）	`v[0] = 10;`
`at()`	下标访问（边界检查）	`v.at(1) = 20;`
`front()`	访问首元素	`int first = v.front();`
`back()`	访问尾元素	`int last = v.back();`
`data()`	返回底层数组指针（[8]）	`int* p = v.data();`

3. 容量操作

函数	说明	示例
`size()`	返回元素个数	`int n = v.size();`
`empty()`	判断是否为空	`if (v.empty()) {...}`
`capacity()`	返回预分配内存大小	`int cap = v.capacity();`
`resize(n)`	调整元素数量（填充默认值）	`v.resize(10);`
`resize(n, val)`	调整元素数量（填充 `val`）	`v.resize(5, -1);`
`reserve(n)`	预分配内存（避免多次扩容）	`v.reserve(100);`

4. 修改操作

函数	说明	示例
`push_back(val)`	尾部添加元素（[2]）	`v.push_back(42);`
`pop_back()`	删除尾部元素	`v.pop_back();`
`insert(pos, val)`	在迭代器 `pos` 前插入元素	`v.insert(v.begin()+2, 99);`
`erase(pos)`	删除迭代器 `pos` 处元素	`v.erase(v.begin());`
`erase(start, end)`	删除 `[start, end)` 区间元素	`v.erase(v.begin(), v.begin()+3);`
`clear()`	清空所有元素	`v.clear();`
`assign(n, val)`	重置为 `n` 个 `val`（[9]）	`v.assign(3, 100);`

5. 迭代器操作

函数	说明	示例
`begin()`	返回首元素迭代器	`auto it = v.begin();`
`end()`	返回尾后迭代器	`for (; it != v.end(); ++it)`
`rbegin()`	返回反向首迭代器（尾部）	`auto rit = v.rbegin();`
`rend()`	返回反向尾后迭代器（首部前）	`while (rit != v.rend())`

6. 其他操作

函数	说明	示例
`swap(v2)`	交换两个 `vector` 内容	`v1.swap(v2);`
`emplace_back(args...)`	尾部就地构造元素（高效版 `push_back`）	`v.emplace_back(1, "text");`

以下是二维 `vector` 的常用操作和函数：

1. 初始化

```
// 空二维vector
vector<vector<int>> matrix;

// 固定大小 (m行n列, 默认值0)
vector<vector<int>> matrix(m, vector<int>(n));

// 固定大小 + 初始值
vector<vector<int>> matrix(m, vector<int>(n, -1));

// 列表初始化
vector<vector<int>> matrix = {{1,2}, {3,4}, {5,6}};
```

2. 元素访问

```
// 下标访问（不检查边界）
int val = matrix[i][j];

// 安全访问（边界检查）
int val = matrix.at(i).at(j);  // [4]

// 访问首行
vector<int>& firstRow = matrix.front();

// 访问尾行
vector<int>& lastRow = matrix.back();  // [6]
```

3. 大小操作

```
// 获取行数
int rows = matrix.size();

// 获取列数（假设矩形）
int cols = matrix.empty() ? 0 : matrix[0].size();

// 检查是否为空
bool isEmpty = matrix.empty();
```

4. 修改操作

```
// 添加新行（末尾）
matrix.push_back({7,8,9});  // [4]

// 删除尾行
matrix.pop_back();

// 修改元素
matrix[i][j] = 42;

// 调整行数
matrix.resize(newRowCount);

// 调整某行列数
matrix[i].resize(newColCount, 0);  // [3]
```

5. 行列操作

```cpp
// 添加元素到第i行
matrix[i].push_back(10);

// 删除第i行末尾元素
matrix[i].pop_back();

// 删除第i行第j列元素
matrix[i].erase(matrix[i].begin() + j);

// 清空特定行
matrix[i].clear();
```

关键注意事项

1. **非矩形支持**：二维 `vector` 每行长度可不同

   ```cpp
   vector<vector<int>> jagged = {{1}, {2,3}, {4,5,6}};
   ```

2. **内存连续性问题**：二维 `vector` 的行在内存中**不连续**。
   - **C++原生二维数组（静态或动态分配）**：在内存中是连续的。无论是静态数组（如`int arr[3][4]`）还是通过一次`new`分配的动态数组（如`int (*arr)[4] = new int[3][4]`），所有元素都存储在一个连续的内存块中，按行优先顺序排列。
   - **二维vector（`vector<vector<T>>`）：不保证内存连续**。外层vector的每个元素是一个内层vector，而每个内层vector独立管理自己的内存。因此，不同行（即不同内层vector）的内存块通常是分开分配的，不连续。同一行内的元素是连续的，但行与行之间不连续。
   - **模拟连续内存的二维结构**：如果想用vector实现内存完全连续的二维数组，可以用一个一维vector（如`vector<int>`）并通过索引计算（`data[i * cols + j]`）来模拟二维访问。